

2.3 Python for EdgeAI Engineers

Edge AI Engineering Bootcamp | Analog Data

Performance-Critical Patterns

Learning Objectives

By the end of this session, students should be able to:

- Use Python generator functions to stream camera frames without holding the entire sequence in memory
 - Architect a producer/consumer pipeline using `threading.Thread` and `queue.Queue` so capture and inference run concurrently
 - Understand why Python's GIL blocks true CPU parallelism and know when to use `multiprocessing` instead
 - Profile code execution down to microsecond precision using `time.perf_counter()`
-

1. Generator Functions for Streaming Frames

A **generator** is a Python function that uses `yield` instead of `return`. Instead of computing all values up front and storing them in a list, it produces one value at a time, on demand. For a camera pipeline that could theoretically run forever, this is the only sane approach — you cannot load thousands of frames into a list before starting inference.

Basic generator pattern

```
def frame_stream(cap):  
    """Yields frames one at a time from an OpenCV VideoCapture object."""  
    while True:  
        ret, frame = cap.read()  
        if not ret:  
            break  
        yield frame
```

Using the generator

```
cap = cv2.VideoCapture(0)  
for frame in frame_stream(cap):  
    result = run_inference(frame)  
    display(result)
```

Why this matters: A plain `for` loop calling `cap.read()` directly works fine for simple demos, but generators allow you to compose pipelines cleanly — you can wrap one generator in another (e.g., a

resize generator wrapping a capture generator) without coupling the two stages together in a single monolithic function.

Composing generators — a real pipeline

```
def resize_stream(frames, size=(320, 240)):
    for frame in frames:
        yield cv2.resize(frame, size)

def normalize_stream(frames):
    for frame in frames:
        yield frame.astype('float32') / 255.0

cap = cv2.VideoCapture(0)
pipeline = normalize_stream(resize_stream(frame_stream(cap)))

for preprocessed_frame in pipeline:
    run_inference(preprocessed_frame)
```

Teaching point: Each stage in the pipeline above processes exactly one frame at a time and hands it forward. No intermediate list exists in memory. This pattern scales from a Pi Zero 2W to a Pi 5 without changing the pipeline architecture.

2. `threading.Thread` + `queue.Queue` — Producer/Consumer for Capture + Inference

The simple generator pipeline above is still **sequential** — capture, then preprocess, then infer, one frame at a time. On a system where capture takes ~10ms and inference takes ~80ms, the camera sits idle for 80ms per frame. The solution is to **decouple capture from inference using threads and a shared queue**.

The producer/consumer pattern

```
[Producer Thread] → queue.Queue → [Consumer Thread]
(camera capture)          (model inference)
```

Implementation

```
import threading
import queue
import cv2

QUEUE_MAXSIZE = 5 # don't let the buffer grow unbounded

def producer(cap, frame_queue):
    """Captures frames and puts them in the queue."""
    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```

    try:
        frame_queue.put_nowait(frame)    # non-blocking; drop frame if queue is
full
    except queue.Full:
        pass    # acceptable: drop stale frames rather than accumulate lag

def consumer(frame_queue, model):
    """Reads frames from the queue and runs inference."""
    while True:
        frame = frame_queue.get()        # blocks until a frame is available
        result = model.infer(frame)
        display(result)
        frame_queue.task_done()

cap = cv2.VideoCapture(0)
frame_queue = queue.Queue(maxsize=QUEUE_MAXSIZE)

t_producer = threading.Thread(target=producer, args=(cap, frame_queue),
daemon=True)
t_consumer = threading.Thread(target=consumer, args=(frame_queue, model),
daemon=True)

t_producer.start()
t_consumer.start()

t_producer.join()
t_consumer.join()

```

Key design decisions to discuss with students

Decision	Why
<code>queue.Queue(maxsize=5)</code>	Caps memory use; prevents the producer from running hundreds of frames ahead of the slow consumer
<code>put_nowait</code> + <code>except queue.Full</code>	Drop frames rather than block capture — stale frames have no value in a real-time system
<code>daemon=True</code>	Threads auto-terminate when the main program exits; prevents zombie threads
<code>queue.get()</code> blocks	The consumer naturally sleeps when the queue is empty, consuming zero CPU while it waits

Teaching point: This pattern — bounded queue, non-blocking producer, blocking consumer — is the template students should reach for whenever they need two stages of a pipeline to run at different speeds. It appears in virtually every production-grade vision AI application on embedded Linux.

3. multiprocessing — Bypassing the GIL for CPU-Bound Inference

Python's **Global Interpreter Lock (GIL)** means that even with multiple threads, only one thread executes Python bytecode at any given moment. For I/O-bound work (waiting for a frame from the camera),

threading works fine — the GIL is released while waiting. But for **CPU-bound** work (heavy NumPy operations, model inference in pure Python), multiple threads still only use one CPU core effectively.

Solution: `multiprocessing` — spawns separate OS processes (each with its own GIL), enabling true parallel CPU use on multi-core Pis.

```
from multiprocessing import Process, Queue

def inference_worker(input_q, output_q, model_path):
    import tf.lite_runtime.interpreter as tflite
    model = tflite.Interpreter(model_path=model_path)
    model.allocate_tensors()
    while True:
        frame = input_q.get()
        result = run_inference(model, frame)
        output_q.put(result)

input_q = Queue(maxsize=4)
output_q = Queue(maxsize=4)

worker = Process(target=inference_worker, args=(input_q, output_q, 'model.tflite'))
worker.start()
```

threading vs multiprocessing — decision guide

Scenario	Recommended
Camera capture (I/O wait)	<code>threading</code> — GIL released during I/O
Serial/network data ingestion	<code>threading</code>
Heavy Python CPU inference	<code>multiprocessing</code>
NumPy-intensive pre/post-processing	<code>multiprocessing</code> (or vectorise with NumPy to avoid pure Python loops)
TFLite/ONNX inference (C-extension based)	Often releases GIL internally — benchmark both before deciding

Teaching point: Many students default to `multiprocessing` for everything because "it bypasses the GIL." Correct this — `multiprocessing` has real overhead: process spawning time, inter-process serialisation (pickling) of frames, separate memory spaces. For a Pi with 1–2GB RAM, spawning many worker processes is expensive. The right choice depends on profiling (see 2.4), not intuition.

4. `time.perf_counter()` — Microsecond Profiling

Before optimising anything, measure it. `time.perf_counter()` is the highest-resolution timer available in Python — suitable for microsecond-level measurements.

```
import time
```

```

start = time.perf_counter()

# --- code under measurement ---
result = model.infer(frame)
# -----

end = time.perf_counter()
elapsed_ms = (end - start) * 1000
print(f"Inference time: {elapsed_ms:.3f} ms")

```

Building a simple frame-rate profiler

```

frame_times = []

for frame in frame_stream(cap):
    t0 = time.perf_counter()
    result = model.infer(frame)
    t1 = time.perf_counter()
    frame_times.append(t1 - t0)

avg_ms = sum(frame_times) / len(frame_times) * 1000
fps = 1.0 / (sum(frame_times) / len(frame_times))
print(f"Average inference: {avg_ms:.2f} ms | Effective FPS: {fps:.1f}")

```

Teaching point: Use `time.perf_counter()`, not `time.time()`. `time.time()` returns wall-clock time with second-resolution on some systems; `time.perf_counter()` uses the highest available hardware counter and is monotonic (it never goes backward due to NTP adjustments or system clock changes).

Hands-on Lab

Lab A — Python loop vs NumPy normalisation benchmark (640×480, 1000 iterations)

Students implement both approaches and use `time.perf_counter()` to measure and compare:

```

import numpy as np, time

frame = np.random.randint(0, 256, (480, 640, 3), dtype=np.uint8)
N = 1000

# Pure Python loop
t0 = time.perf_counter()
for _ in range(N):
    result = [[px / 255.0 for px in row] for row in channel] for channel in
frame]
t1 = time.perf_counter()
print(f"Python loop: {(t1-t0)*1000:.1f} ms total")

# NumPy vectorised
t0 = time.perf_counter()
for _ in range(N):
    result = frame.astype('float32') / 255.0

```

```
t1 = time.perf_counter()
print(f"NumPy vectorised: {(t1-t0)*1000:.1f} ms total")
```

Expected finding: NumPy is typically 100× or more faster. Students should document the ratio and explain *why* in terms of NEON SIMD (from Module 1.1) and the GIL.

Lab B — Thread-safe frame ring buffer

Students implement a ring buffer using `threading.Thread` + `queue.Queue` where the producer is a camera capture loop and the consumer prints the frame shape and timestamp. They must demonstrate it runs without deadlocking and handles queue-full conditions gracefully by dropping frames.

Summary Checklist for Students

- ☐ Can write a generator function that yields frames from a camera indefinitely
- ☐ Can compose two or more generator stages into a clean preprocessing pipeline
- ☐ Can implement a bounded producer/consumer using `threading.Thread` and `queue.Queue`
- ☐ Can explain when to use `multiprocessing` vs `threading` and why the GIL matters
- ☐ Can use `time.perf_counter()` to profile any code block to millisecond precision
- ☐ Has measured the Python-loop vs NumPy vectorisation speed difference empirically